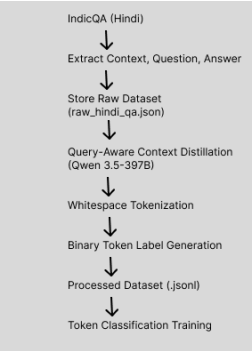
DSAI Project Milestone 5

Trained Model and Pipeline

The previous milestone focused on developing a prompt compression model for the Indic Question Answering (IndicQA) and x task using a token classification approach. The objective was to identify and retain only the most informative tokens from an input prompt while preserving the semantic information necessary for downstream question answering. Instead of generating compressed prompts directly, the problem was formulated as a sequence labeling task in which each token was assigned a binary label indicating whether it should be retained or discarded.



The training pipeline began with the preparation of the IndicQA dataset, where each training example consisted of a question and its corresponding context. Compressed prompts generated using a large language model were aligned with the original prompts through the Query-Aware Distillation and Alignment process, producing binary token-level labels. These labels served as the ground truth for supervised learning.

The prepared data was then tokenized using the XLM-RoBERTa tokenizer, and the binary labels were aligned with the resulting subword tokens to ensure correct supervision during training. The processed dataset was subsequently used to fine-tune an XLM-RoBERTa-based Token Classification model, enabling it to predict whether each token should be retained in the compressed prompt.

During training, the model was optimized using supervised learning and evaluated using Precision, Recall, F1-score, and Accuracy, with particular emphasis on the F1-score due to the token classification nature of the task. The trained model from the previous milestone established the baseline prompt compression system, providing the foundation for the current milestone, which focuses on improving model performance through systematic hyperparameter optimization and training refinements.

Evaluation Dataset and Preprocessing

The evaluation of the prompt compression model was performed using the **validation** and **test** splits of the prepared IndicQA dataset. The notebook loads the dataset from three JSONL files—train.jsonl (1610 samples), val.jsonl (202 samples), and test.jsonl (203samples) - ensuring that model training, validation, and final testing are conducted on separate data partitions. Each sample in the dataset contains the following fields:

- **question** – the input question.
- **original_answer** – the corresponding answer from the dataset.
- **tokens** – the tokenized representation of the input sequence.
- **labels** – binary token-level labels indicating whether each token should be retained (1) or discarded (0) during prompt compression.

During evaluation, the notebook uses the **validation split** for model selection and hyperparameter tuning, while the **test split** is reserved for the final performance assessment of the trained model.

Before evaluation, the same preprocessing pipeline used during training is applied to the validation and test datasets to ensure consistency. The preprocessing steps include:

1. Tokenizing the input using the **XLM-RoBERTa tokenizer**.
2. Aligning the binary token labels with the generated subword tokens.
3. Assigning an **ignore label (-100)** to padding tokens and continuation subword tokens so that they do not contribute to the evaluation metrics.
4. Applying dynamic padding through the data collator to create uniformly sized batches for efficient inference.

Model performance is then computed only on valid token positions by excluding all tokens assigned the ignore label. The evaluation reports **Accuracy, Precision, Recall, and F1-score**, with the **F1-score** serving as the primary metric for selecting the best-performing model during training.

Evaluation Environment, Hardware, Software Frameworks, and Runtime Setup

The evaluation was performed in the same computational environment used for model training to ensure consistency and reproducibility of results. All experiments were executed on a cloud-based Linux runtime equipped with **dual NVIDIA Tesla T4 GPUs**, each providing **15 GB of GPU memory**, running with **CUDA 13.0** and **NVIDIA driver version 580.159.04**. GPU

availability was verified before model loading, ensuring that all evaluation tasks utilized hardware acceleration for efficient inference and metric computation.

The implementation was developed using the **PyTorch** deep learning framework together with the **Hugging Face Transformers** library for model loading, tokenization, and inference. Dataset preparation and management were performed using the **Hugging Face Datasets** library, while evaluation metrics were computed using the **Evaluate** library. Additional libraries, including **NumPy**, **Pandas**, **Scikit-learn**, and **Matplotlib**, were used for data preprocessing, analysis, visualization, and performance reporting.

To ensure reproducibility, a fixed random seed was initialized before training and evaluation, and the same tokenizer, preprocessing pipeline, and label alignment strategy were applied consistently across all datasets. The runtime employed mixed-precision (FP16) computation whenever supported by the hardware to improve computational efficiency and reduce GPU memory usage. Dynamic padding through a data collator minimized unnecessary computation during batch processing, while model checkpoints, training logs, and configuration files were stored throughout the experiments. These settings ensure that the evaluation can be reproduced under identical software, hardware, and runtime configurations using the same hyperparameters and preprocessing pipeline

Performance Metrics Used for Evaluation

The model was evaluated using **Accuracy, Precision, Recall, and F1-score**, which were computed through a custom compute_metrics function implemented using the **Scikit-learn** metrics library. During evaluation, the model first converts the predicted logits into class labels using the **argmax** operation. Tokens with the label value **-100** are excluded from evaluation, ensuring that padding tokens and continuation subword tokens introduced during tokenization do not influence the reported performance. This produces a fair evaluation based only on meaningful token predictions.

The evaluation computes **binary Precision, Recall, and F1-score** using precision_recall_fscore_support(average="binary"), indicating that the task focuses on distinguishing important tokens from non-important tokens. **Precision** measures how many tokens predicted as important are actually correct, while **Recall** measures how many of the truly important tokens are successfully identified by the model. The **F1-score**, which balances precision and recall, is the primary evaluation metric and is also used by the training pipeline to select the best-performing model (metric_for_best_model="f1"). This makes F1-score particularly appropriate because the objective of the task is to accurately identify relevant tokens while minimizing both missed predictions and false positives. **Accuracy** is reported as an additional metric to provide an overall measure of token-level prediction correctness, although it

is not used as the model selection criterion due to the greater importance of balancing precision and recall in this binary token classification task.

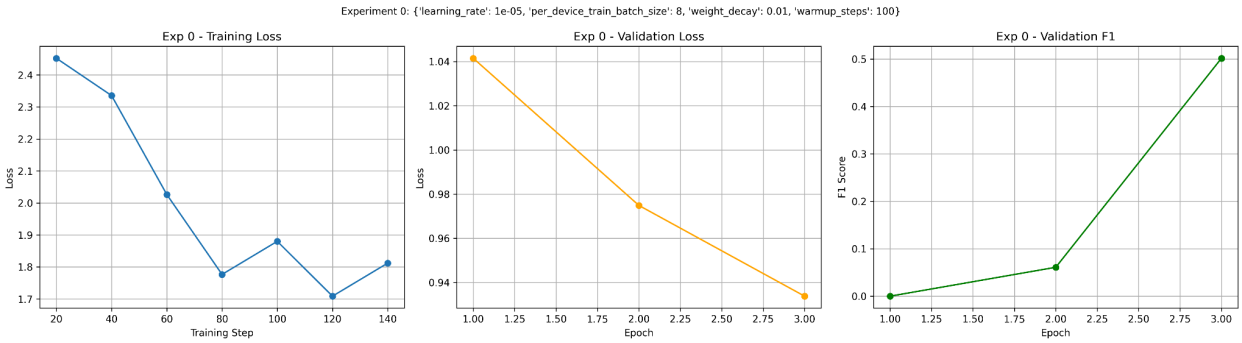
Quantitative Results

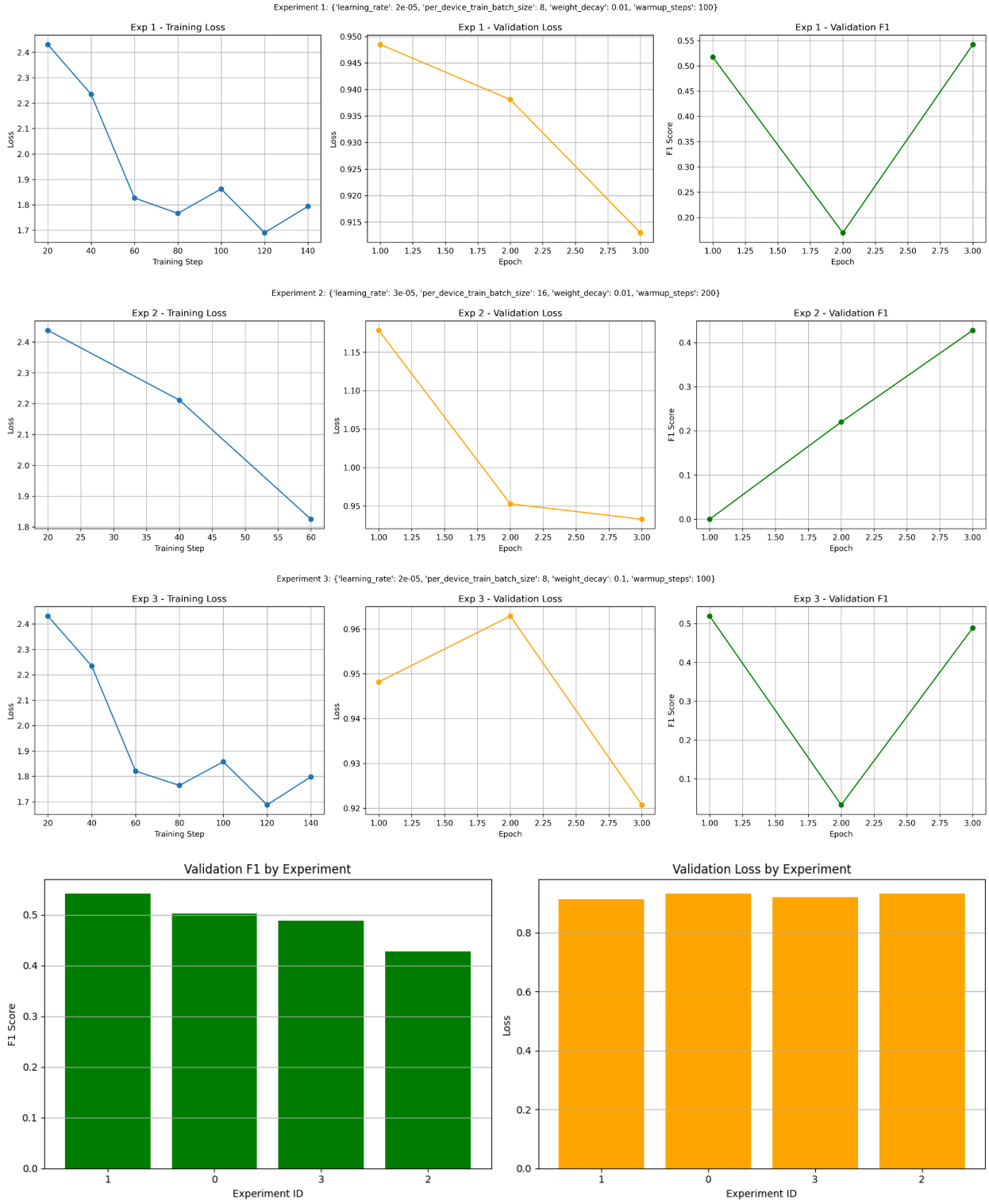
Hyperparameter Results Comparison

The performance of the XLM-RoBERTa token classification model was evaluated after training under different hyperparameter configurations. Each experiment was conducted independently using the same training and validation datasets while varying selected hyperparameters. The validation set was used to compare different configurations and identify the model that achieved the best overall performance.

Model performance was evaluated using Accuracy, Precision, Recall, and F1-score, with the F1-score serving as the primary criterion for selecting the best-performing configuration. The notebook also records the training loss and validation loss throughout the training process, enabling analysis of convergence behaviour and model generalization.

Experiment	Learning rate	per_device_train_batch_size	weight_decay	Warmup steps	Validation Accuracy	Validation Precision	Validation Recall	Validation F1
0	1e-05	8	0.01	100	0.73	0.54	0.46	0.50
1	2e-05	8	0.01	100	0.74	0.55	0.53	0.54
2	3e-05	16	0.01	200	0.72	0.51	0.36	0.42
3	2e-05	8	0.1	100	0.74	0.57	0.42	0.48





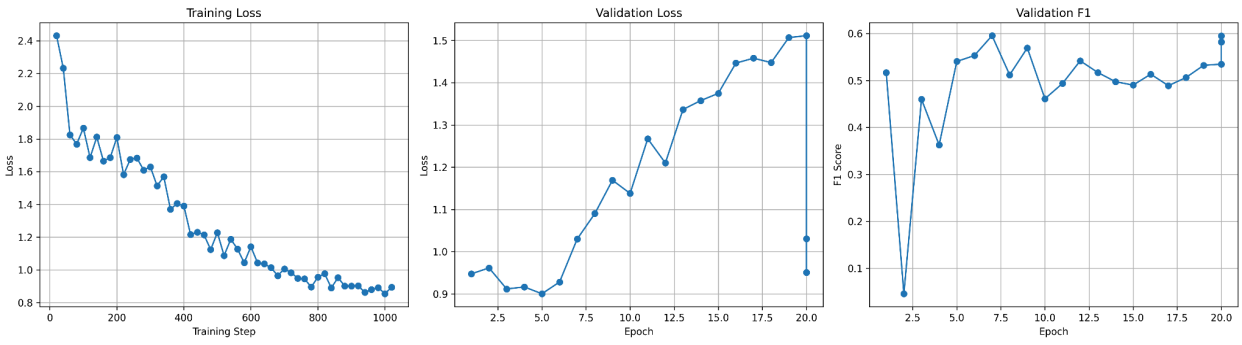
The experimental results show that all configurations achieved a reduction in training and validation loss across epochs, indicating successful model learning and convergence. Among the four experiments, **Experiment 1**(i.e exp id =1) achieved the highest validation F1-score (approximately **0.54**) while maintaining a low validation loss, making it the best-performing

configuration overall. Although Experiments 0 and 3 produced comparable validation losses, their F1-scores were slightly lower, indicating that lower loss alone did not necessarily translate into better token classification performance. Experiment 2 recorded the lowest validation F1-score (approximately **0.43**) despite a similar validation loss, suggesting that its hyperparameter configuration was less effective at balancing precision and recall. Overall, the comparative analysis indicates that the hyperparameter settings used in **Experiment 1** provide the best trade-off between convergence and predictive performance, and were therefore selected as the optimal configuration for the final model.

Training Summary (Best model selected)

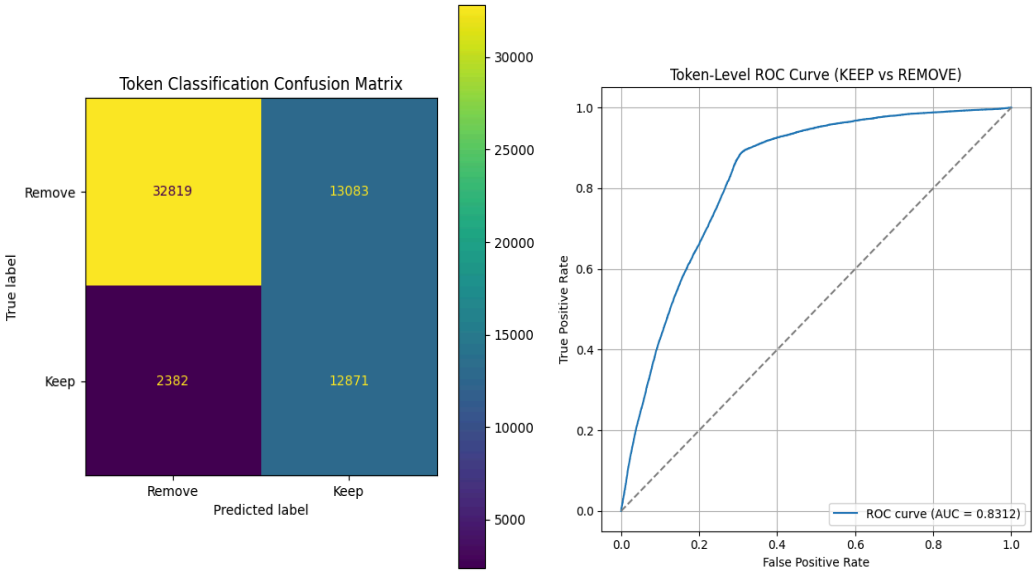
Parameter	Value
Learning Rate	2e-05
Batch Size	8
Weight decay	0.01
Warmup steps	100
Number of epochs	20
Best Validation F1	0.59

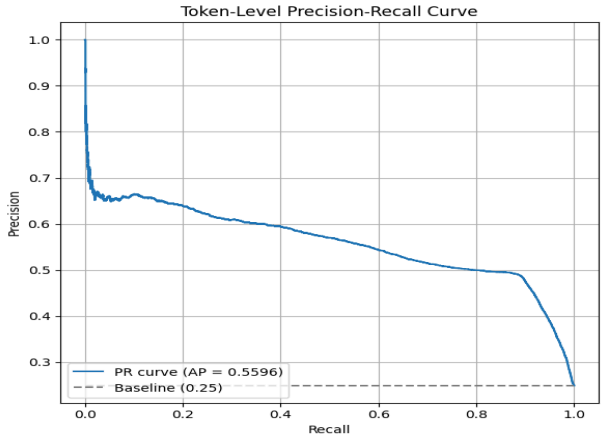
Final Model Performance on Evaluation set



Metric	Score
Loss	1.03
Accuracy	0.73
Precision	0.53
Recall	0.67
F1-score	0.59

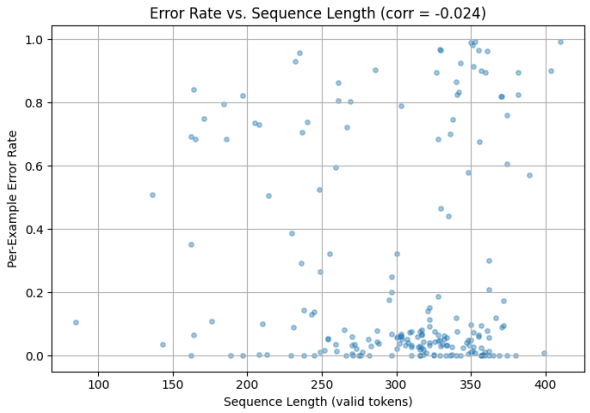
Task-Specific Evaluation Visualizations

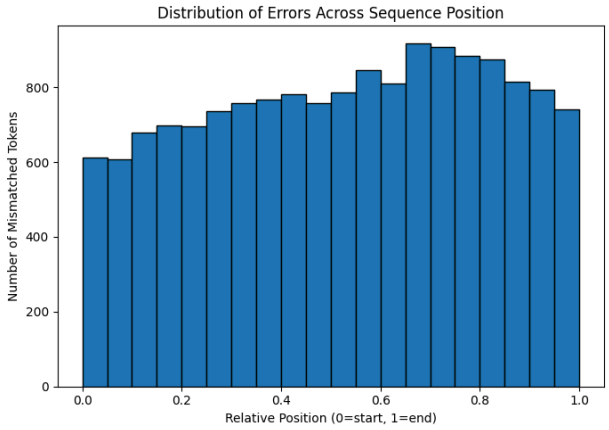




The confusion matrix shows that the model correctly classifies a large proportion of both Remove and Keep tokens, with relatively fewer misclassifications, indicating good token-level prediction performance. The ROC curve achieves an AUC of approximately 0.83, demonstrating strong discrimination between the two classes across different decision thresholds. The Precision–Recall curve reports an Average Precision (AP) of approximately 0.60, indicating that the model maintains a reasonable balance between precision and recall for the positive (Keep) class. The gradual decline in precision as recall increases suggests the expected trade-off when identifying more relevant tokens. Overall, these evaluation plots indicate that the model generalizes well and provides reliable performance for the binary token classification task, with good discriminative ability and balanced predictive accuracy

Qualitative Results: Successful Predictions and Failure Cases





Token	Errors	Occurrences	Error Rate
_सोना	8	8	100.00%
_नित्य	7	7	100.00%
_सेक्स	6	6	100.00%
_देखी	6	6	100.00%
_विश्वकप	6	6	100.00%
_बनाकर	5	5	100.00%
_वित्तीय	5	5	100.00%
_टीवी	14	15	93.33%
_फ़िल्म	19	21	90.48%
_मां	6	7	85.71%
_गेंद	6	7	85.71%
_चिंता	6	7	85.71%
_जाँच	10	12	83.33%
_पारी	5	6	83.33%
_अरुण	5	6	83.33%
_गिरावट	5	6	83.33%
_चट	5	6	83.33%
_नाका	5	6	83.33%
_संवाददाता	9	11	81.82%
_ठीक	9	11	81.82%

The qualitative analysis indicates that the model performs well on the majority of evaluation samples, correctly identifying **Keep** and **Remove** tokens, as reflected by the low error rates observed for most sequences. The error rate versus sequence length plot shows a **correlation of**

-0.024, indicating that prediction errors are not strongly influenced by the input sequence length. This suggests that the model maintains relatively consistent performance across both shorter and longer sequences, with most samples exhibiting low error rates despite variations in sequence length.

The analysis of errors across sequence positions reveals that misclassifications occur throughout the sequence but are slightly more concentrated in the latter half (approximately **60%–90%** of the relative sequence length). This indicates that the model is generally robust across the input but encounters greater difficulty when predicting tokens appearing toward the end of longer contexts.

The token-level error analysis highlights several failure cases. Tokens such as **"_सोना", "_नित्य", "_सेक्स", "_देखी", "_विवेककुम", "_बनाकर", and "_वित्तीय"** exhibit an **error rate of 100%**, meaning they were misclassified in every occurrence within the evaluation set. Other challenging tokens include **"_टीवी" (93.33%)**, **"_फ़िल्म" (90.48%)**, **"_माँ" (85.71%)**, **"_गेंद" (85.71%)**, and **"_पिता" (85.71%)**, indicating that these specific words consistently posed difficulties for the model. These results suggest that the remaining prediction errors are concentrated around a small subset of tokens rather than being uniformly distributed across the dataset, providing clear targets for future improvements in data coverage and model refinement.

Model Limitations

The evaluation results indicate that the model still struggles with predicting certain tokens consistently. Token-level error analysis shows that several low-frequency tokens, such as **"_सोना", "_नित्य", "_सेक्स", "_देखी", "_विवेककुम", "_बनाकर", and "_वित्तीय"**, were misclassified in all of their occurrences, resulting in a **100% error rate**. Additionally, tokens such as **"_टीवी" (93.33%)**, **"_फ़िल्म" (90.48%)**, and **"_माँ" (85.71%)** also exhibited high error rates, suggesting that the model has difficulty generalizing to specific lexical patterns. The overall **Validation F1-score of approximately 0.53** further indicates that there is room for improvement in balancing precision and recall for the binary token classification task.

Observed Anomalies

The evaluation revealed that prediction errors are **not correlated with input sequence length**, as indicated by the correlation coefficient of **-0.024** between sequence length and error rate. This suggests that increasing sequence length does not significantly degrade model performance. However, the distribution of errors across sequence positions shows that misclassifications are slightly more concentrated in the **latter portion of the sequence (approximately 60%–90% of the relative sequence length)**, indicating that the model finds later contextual tokens comparatively more challenging. Furthermore, although multiple experiments achieved nearly identical validation losses, their validation F1-scores differed noticeably, demonstrating that lower loss did not always correspond to better token classification performance.

Gap Between Expected and Actual Performance

The objective of the model was to accurately identify **Keep** and **Remove** tokens with high precision and recall across all token types. While the model achieved a **ROC-AUC of approximately 0.83**, demonstrating good discriminative capability, the **Average Precision of approximately 0.60** and **Validation F1-score of approximately 0.53** indicate that prediction quality remains moderate. The confusion matrix also shows that although most tokens are classified correctly, a noticeable number of **false positives and false negatives** are still present. These findings indicate that the model performs well on the majority of samples but requires further improvements to better handle difficult token categories and achieve more balanced token-level predictions.